

FREDERICO LUIZ STREY

PIPELINE DE CONTAGEM DE PARAFUSOS POR VISÃO COMPUTACIONAL

Relatório Técnico

Vitória - ES,
31 de maio de 2026

Sumário

1	Introdução	2
1.1	Requisitos, Restrições e Justificativa da Abordagem	2
1.2	Visão Geral do Pipeline	3
2	Etapa 1 — Conversão para Escala de Cinza	4
3	Etapa 2 — Suavização com Filtro Gaussiano	5
4	Etapa 3 — Binarização Adaptativa	6
5	Etapa 4 — Operações Morfológicas	7
6	Etapa 5 — Algoritmo Watershed	8
7	Etapa 6 — Remoção de Bordas	10
8	Etapa 7 — Detecção de Contornos	11
9	Etapa 8 — Filtros por Métricas de Forma	11
9.1	Área (AREA_MIN e AREA_MAX)	12
9.2	Solidez (SOLIDEZ_MIN)	13
9.3	Razão de Aspecto (ASPECTO_MIN e ASPECTO_MAX)	13
9.4	Extent (EXTENT_MIN)	14
10	Conclusão	15
11	Guia de Diagnóstico — O que fazer quando a contagem erra	17
11.1	Cenário A: Parafusos válidos sendo rejeitados	17
11.2	Cenário B: Ruídos e falsos positivos sendo contados	18
11.3	Cenário C: Parafusos agrupados contados como um só	18
11.4	Cenário D: Parafuso único dividido em dois	19
12	Aplicação Web — Microserviço FastAPI	19

1 Introdução

1.1 Requisitos, Restrições e Justificativa da Abordagem

O desenvolvimento da solução foi realizado sob um conjunto de restrições técnicas e operacionais que influenciaram diretamente a escolha metodológica adotada. Um dos principais requisitos do desafio era a possibilidade de execução em dispositivos com recursos computacionais limitados, preferencialmente em ambiente móvel. Essa restrição inviabiliza o uso de modelos de segmentação de grande porte e alto custo computacional, como arquiteturas modernas baseadas em Transformers para visão computacional, a exemplo da família Segment Anything Model (1), que normalmente requer aceleração por GPU e elevada quantidade de memória para operação eficiente.

Outro fator limitante foi a disponibilidade reduzida de dados. O conjunto fornecido continha apenas cinco imagens para análise e calibração da solução. Embora suficiente para o desenvolvimento e ajuste manual de um pipeline clássico, essa quantidade é insuficiente para treinamento, validação e avaliação robusta de modelos supervisionados mais complexos, como redes neurais convolucionais (2) voltadas para detecção ou segmentação de objetos. Além disso, a escassez de amostras dificulta a obtenção de métricas estatisticamente representativas, aumentando o risco de sobreajuste e reduzindo a capacidade de generalização de modelos aprendidos.

Diante dessas restrições, optou-se pela utilização exclusiva de técnicas clássicas de Visão Computacional. Essa abordagem apresenta baixo custo computacional, alta interpretabilidade e não depende de conjuntos extensos de dados rotulados para treinamento. O pipeline proposto emprega sucessivas etapas de pré-processamento para redução de ruído, compensação de variações de iluminação, mitigação de reflexos e segmentação dos objetos de interesse. Em seguida, são extraídos contornos e calculados descritores geométricos capazes de caracterizar a forma dos objetos identificados.

Por fim, uma série de filtros baseados em propriedades geométricas, como área, solidez, razão de aspecto e extent, é aplicada para distinguir parafusos válidos de artefatos, sombras, reflexos e outros falsos positivos presentes na imagem. Dessa forma, a solução busca maximizar a robustez da contagem mantendo compatibilidade com ambientes de execução restritos e com disponibilidade limitada de dados para desenvolvimento.

O pipeline transforma uma fotografia colorida de parafusos numa contagem numérica por meio de oito etapas sequenciais de processamento digital de imagens (3). Cada etapa prepara a imagem para a seguinte, refinando progressivamente a representação até que objetos individuais possam ser identificados, isolados e classificados utilizando as estruturas nativas da biblioteca OpenCV (4).

1.2 Visão Geral do Pipeline

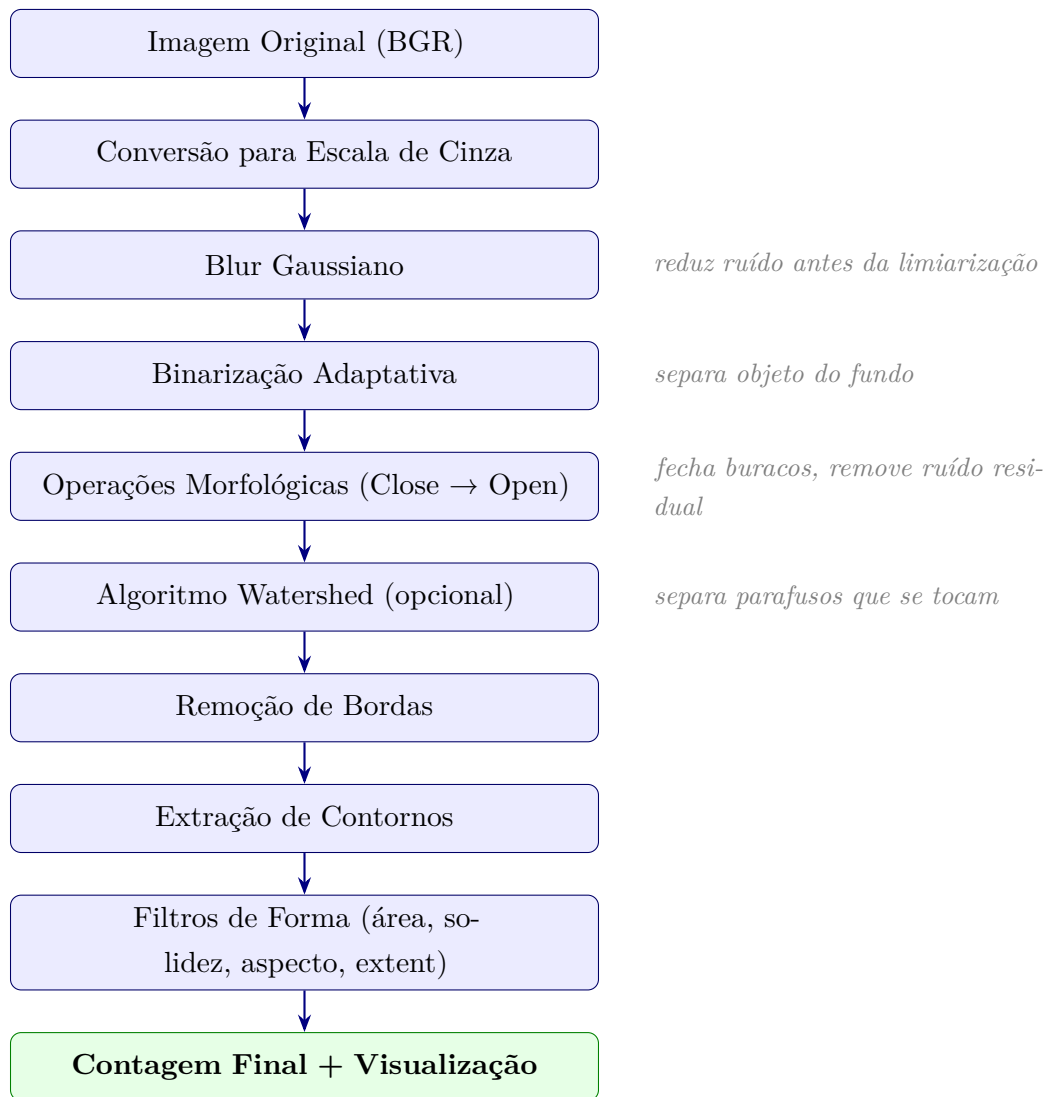


Figura 1: Diagrama sequencial do pipeline de contagem de parafusos.

2 Etapa 1 — Conversão para Escala de Cinza

O que faz

Colapsa os três canais de cor (R, G, B) num único canal de intensidade luminosa. A fórmula padrão do OpenCV pondera os canais de acordo com a percepção visual humana estabelecida pelas recomendações de luminância do (author?) (5).

Por que é importante

Todos os algoritmos subsequentes (limiarização, morfologia, transformada de distância) operam em imagens de um único canal (6). Trabalhar em escala de cinza reduz o volume de dados em $3\times$ e simplifica os cálculos sem perder as informações estruturais relevantes — contornos, texturas e contrastes de brilho — que permitem distinguir parafusos do fundo.

Como ajustar

Não há parâmetro configurável nesta etapa. Se a imagem original possuir um canal de cor com contraste particularmente alto entre o parafuso e o fundo (por exemplo, em fotografia com iluminação colorida), pode valer a pena utilizar apenas esse canal isolado em vez da conversão padrão. O método `cv2.split(imagem_bgr)` retorna os canais B, G e R separados para inspeção individual.

Etapa 0 — Imagem Original



Figura 2: Imagem Original.

Etapa 1 — Escala de Cinza



Figura 3: Escala Cinza.

3 Etapa 2 — Suavização com Filtro Gaussiano

Parâmetro

```
1 BLUR_KERNEL = (5, 5)
```

O que faz

Aplica uma convolução com um *kernel* gaussiano bidimensional sobre a imagem em escala de cinza (7). Cada pixel recebe um valor médio ponderado dos pixels vizinhos, onde vizinhos mais próximos têm peso maior, modelado pela distribuição normal estrutural. O resultado é uma imagem “borrada” que elimina variações abruptas de intensidade causadas por ruído de sensor, grão fotográfico e pequenas imperfeições de superfície.

Por que é importante

Sem suavização, pequenas variações de brilho dentro do corpo de um parafuso geram bordas falsas durante a binarização. Isso fragmenta um único parafuso em dezenas de regiões desconexas, inviabilizando a contagem. O *blur* de pré-processamento é a principal defesa clássica contra ruídos de alta frequência (3).

Como ajustar

O *kernel* deve ser sempre uma tupla de dois valores **ímpares** e **iguais** (o OpenCV exige dimensões ímpares para centrar corretamente o *kernel*).

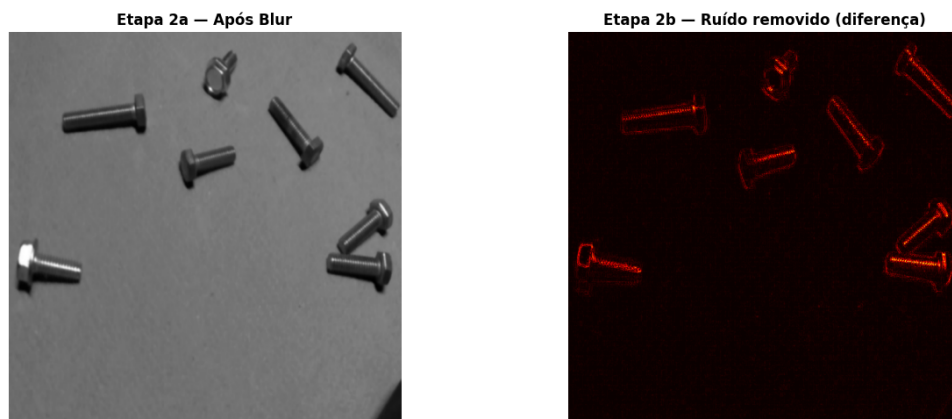


Figura 4: Remoção de Ruído com Blur Gaussiano.

Regra prática: aumente o *kernel* se a binarização resultar em muitos fragmentos pequenos dentro dos parafusos. Diminua se detalhes importantes de borda estiverem sendo perdidos, fazendo parafusos fundirem com o fundo.

4 Etapa 3 — Binarização Adaptativa

Parâmetros internos

```
1 blockSize = 51
2 C          = 4
3 method     = cv2.ADAPTIVE_THRESH_GAUSSIAN_C
4 type       = cv2.THRESH_BINARY_INV
```

O que faz

Converte a imagem em escala de cinza numa imagem binária (preto e branco puro). Para cada pixel, é calculado um limiar **local** baseado nos pixels vizinhos dentro de uma janela de tamanho `blockSize × blockSize`, utilizando a técnica de limiarização local adaptativa. O limiar é a média gaussiana ponderada da vizinhança menos a constante C . Pixels mais escuros que o limiar tornam-se brancos (255); os mais claros tornam-se pretos (0). O sufixo `_INV` inverte esse resultado — objetos escuros sobre fundo claro ficam brancos na imagem binária.

Por que a limiarização adaptativa é superior à global

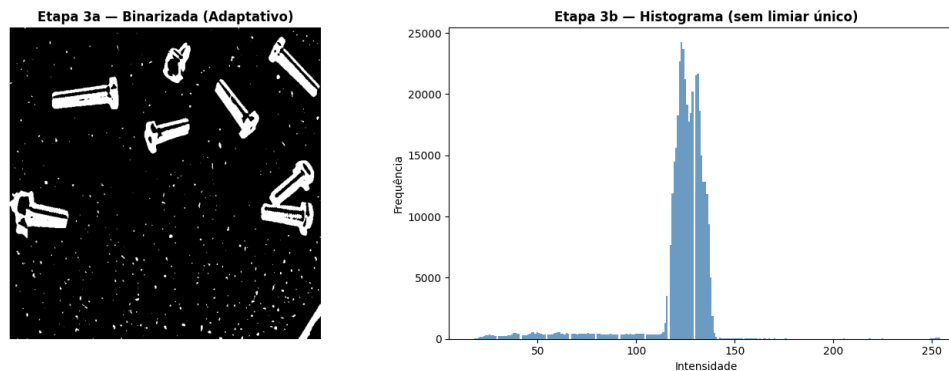
Fotografias de parafusos raramente possuem iluminação uniforme. Uma parte da imagem pode estar mais iluminada, outra em sombra. Um limiar global único, como o célebre método de Otsu (8), funcionaria bem na região bem iluminada, mas falha nas sombras, parafusos escurecidos se fundem com o fundo. A limiarização adaptativa resolve isso calculando um limiar diferente para cada região, compensando automaticamente as variações locais de iluminação (4).

Como ajustar `blockSize`

`blockSize` define o tamanho da vizinhança usada para calcular o limiar local. Deve ser um número **ímpar** e maior que o tamanho dos detalhes de textura que se deseja ignorar.

Como ajustar `C`

C é subtraído do valor médio calculado antes de compará-lo com o pixel. Valores maiores de C tornam o limiar mais alto, resultando em menos pixels classificados como objeto. Valores menores ou negativos incluem mais pixels como objeto.



5 Etapa 4 — Operações Morfológicas

Parâmetros

```
1 CLOSE_KERNEL = (9, 9)      # tamanho do kernel da operacao Close
2 CLOSE_ITER    = 1          # numero de iteracoes do Close
3 MORPH_KERNEL  = (3, 3)     # tamanho do kernel da operacao Open
4 MORPH_ITER    = 2          # numero de iteracoes do Open
```

Conceito fundamental: erosão e dilatação

Todas as operações da Morfologia Matemática baseiam-se na teoria dos conjuntos aplicada a estruturas geométricas (9):

- **Dilatação:** expande regiões brancas — buracos pequenos somem, bordas crescem.
- **Erosão:** encolhe regiões brancas — manchas pequenas somem, bordas recuam.

Operação *Close* (Fechamento)

Sequência: Dilatação $\rightarrow$ Erosão.

O fechamento preenche **buracos internos** e **lacunas pequenas** dentro dos objetos brancos sem alterar significativamente o contorno externo (10). No contexto dos parafusos, a cabeça e o corpo frequentemente aparecem com pequenos buracos escuros (reflexos, texturas de rosca) após a binarização. O *Close* une essas partes num *blob* contínuo.

```
1 kernel_close = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, CLOSE_KERNEL
    )
2 fechada = cv2.morphologyEx(binaria, cv2.MORPH_CLOSE, kernel_close,
3                               iterations=CLOSE_ITER)
```


Operação *Open* (Abertura)

Sequência: Erosão $\rightarrow$ Dilatação.

Remove **manchas pequenas e ruídos** isolados sem alterar objetos maiores (3).

Ordem importa: aplique *Close* antes do *Open*. O *Close* preenche buracos primeiro (consolida o objeto); o *Open* depois remove ruídos externos sem desfazer o que o *Close* construiu.

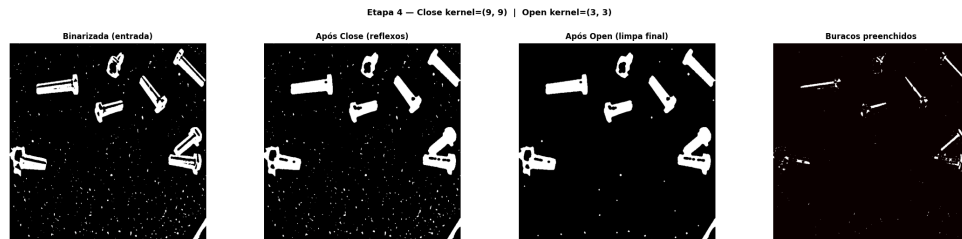


Figura 6: Imagem após Close/Open.

6 Etapa 5 — Algoritmo Watershed

Parâmetros

```

1 USAR_WATERSHED          = True      # ativa/desativa o algoritmo
2 WATERSHED_DIST_LIMIAR   = 0.3      # fracao do maximo da transformada de
   distancia

```

O problema que o Watershed resolve

Quando dois ou mais parafusos estão em contato ou sobrepostos, operações morfológicas os fundem num único *blob*. Sem separação, todo o grupo seria contado como um único parafuso. O algoritmo Watershed baseado em marcadores é o método clássico e topológico ideal para segmentar objetos adjacentes (11).

Como funciona (passo a passo)

1. **Transformada de Distância:** para cada pixel branco na máscara binária, calcula a distância Euclidiana até o pixel preto mais próximo (12). O resultado é uma “topografia” — pixels no centro de um objeto estão no “topo da montanha”; pixels na borda estão na “base”.
2. **Definição do *foreground* seguro:** aplica um limiar sobre a transformada de distância. Apenas pixels com distância maior que $\text{WATERSHED_DIST_LIMIAR} \times d_{\max}$ são considerados “centro com certeza” de um objeto.

3. **Definição do *background* seguro:** dilata levemente a máscara para garantir uma área ao redor dos objetos que é definitivamente fundo.
4. **Zona desconhecida:** pixels entre o *foreground* e *background* seguros — a região de fronteira ambígua onde a separação acontece.
5. **Componentes conectados:** rotula cada região de *foreground* seguro com um ID único — esses são os “marcadores” iniciais (13).
6. **Inundação:** o Watershed “inunda” a topografia a partir de cada marcador. Quando duas frentes de inundação se encontram, uma linha divisória (dique) é traçada. Esta é a separação exata entre os parafusos.

Como ajustar WATERSHED_DIST_LIMIAR

Tabela 1: Guia de ajuste do limiar da transformada de distância.

Valor	Efeito	Quando usar
0,1–0,2	Limiar baixo; mais pixels são “centro seguro”	Parafusos pequenos; pode sobre-segmentar
0,3–0,4	Equilíbrio padrão	Caso geral
0,5–0,6	Limiar conservador; apenas núcleos centrais	Parafusos grandes com muito espaço interno
0,7+	Muito conservador; risco de não detectar núcleos	Evitar, salvo casos extremos

Regra prática:

- Se objetos que *não deveriam ser separados* estão sendo divididos, **aumente** o limiar (ex.: $0,3 \rightarrow 0,5$).
- Se parafusos que *deveriam ser separados* continuam fundidos, **diminua** o limiar (ex.: $0,3 \rightarrow 0,2$).

Quando desativar o Watershed

- Imagens com parafusos bem separados entre si.
- Imagens com muito ruído onde o Watershed cria centenas de micro-segmentos.
- Fase inicial de ajuste de parâmetros — desative para entender primeiro o comportamento base do pipeline.

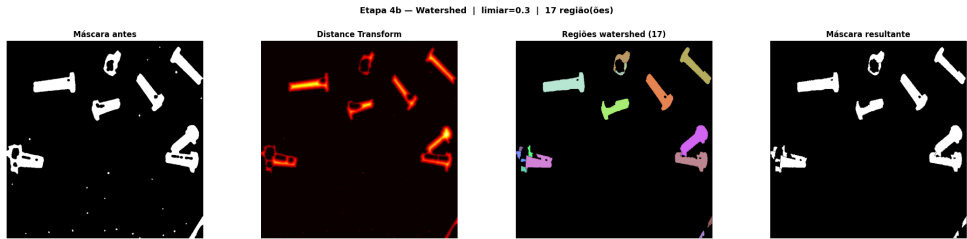


Figura 7: Filtros Watershed.

7 Etapa 6 — Remoção de Bordas

Parâmetro

```
1 MARGEM = 30 # largura em pixels da faixa de borda a zerar
```

O que faz

Após o processamento morfológico, as quatro bordas da imagem são zeradas (pixels definidos como preto): os primeiros e últimos **MARGEM** pixels em cada direção. Isso elimina objetos parcialmente visíveis que foram cortados pelo enquadramento da fotografia.

Por que é importante

Um parafuso cortado pela borda da imagem teria forma incompleta — ele passaria pelos filtros de área (menor que o esperado) ou seria rejeitado pelos filtros de forma (aspecto e solidez distorcidos pela parte faltante). Além disso, artefatos de processamento que ocorrem tipicamente nas margens (efeitos de borda da convolução, reflexos de *flash*) entrariam nos filtros e gerariam falsos positivos (6).

Tabela 2: Guia de ajuste da MARGEM.

Valor (px)	Efeito	Quando usar
10–15	Margem mínima	Parafusos muito próximos às bordas
20–30	Padrão	Caso geral
50+	Margem larga	Fotografias com reflexos ou bordas escuras acentuadas

Atenção: se parafusos válidos estão próximos às bordas, uma margem muito grande vai descartá-los. Nesse caso, o correto é ajustar o enquadramento da câmera para que os parafusos fiquem centralizados, ou reduzir a margem.

8 Etapa 7 — Detecção de Contornos

O que faz

O algoritmo `cv2.findContours` percorre a imagem binária estruturada e identifica as fronteiras externas de cada região mapeada, implementando o clássico algoritmo de varredura topológica de (author?) (14). O retorno consiste em uma lista de contornos, onde cada contorno é armazenado como um vetor sequencial de pontos (x, y) .

Os parâmetros utilizados são:

- **RETR_EXTERNAL:** retorna apenas o contorno externo de cada *blob* (ignora buracos internos estruturais).
- **CHAIN_APPROX_SIMPLE:** comprime segmentos horizontais, verticais e diagonais, armazenando apenas os pontos extremos — economiza memória e tempo computacional de forma robusta (4).

Por que é importante

Os contornos estruturados são a base matemática indispensável para extração de descritores de forma geométricas (7), usados nos filtros da etapa seguinte. Sem eles, seria impossível mensurar métricas determinísticas como área tridimensional plana, solidez estrutural ou aspecto de cada candidato.

9 Etapa 8 — Filtros por Métricas de Forma

Esta é a etapa de classificação: cada contorno detectado é avaliado segundo quatro critérios independentes de geometria analítica espacial. Um contorno precisa **passar em todos** para ser contado como parafuso válido.

Etapa 5 — Todos os Contornos (7)



Figura 8: Contornos encontrados.

9.1 Área (AREA_MIN e AREA_MAX)

```
1 AREA_MIN = 200 % pixels^2
2 AREA_MAX = 50_000 % pixels^2
```

O que é: a área é o número absoluto de pixels contidos nos limites do contorno, calculada computacionalmente através do Teorema de Green por meio da função `cv2.contourArea()` (4). É a métrica de magnitude mais simples e geralmente a mais discriminativa do pipeline.

Por que é importante:

- **AREA_MIN** elimina ruídos, sujeira, reflexos minúsculos e fragmentos morfológicos residuais.
- **AREA_MAX** descarta regiões enormes que claramente não são parafusos individuais — sombras, reflexos grandes, aglomerações não separadas pelo Watershed.

Procedimento recomendado:

1. Ative `MODO_DEBUG = True` e processe uma imagem representativa.
2. Observe a visualização final — contornos rejeitados mostram o motivo (ex.: `area=150`).
3. Anote a faixa de área dos parafusos válidos e dos falsos positivos.
4. Defina `AREA_MIN` ligeiramente abaixo do menor parafuso válido e `AREA_MAX` ligeiramente acima do maior.

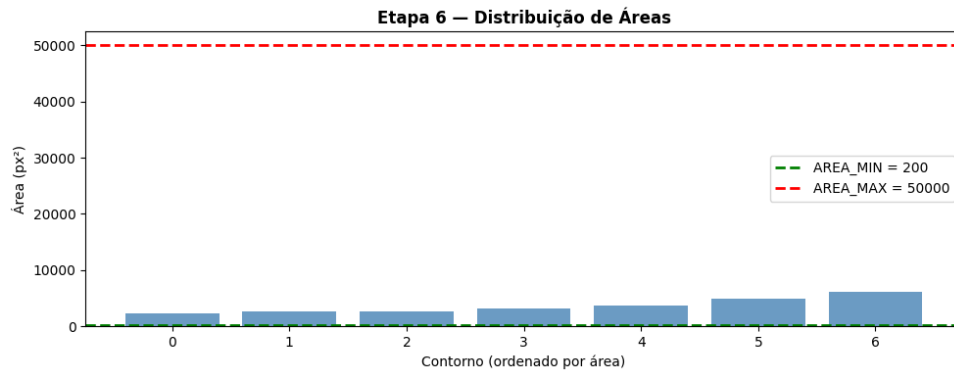


Figura 9: Distribuição de Áreas de cada Contorno Candidato.

9.2 Solidez (`SOLIDEZ_MIN`)

```
1 SOLIDEZ_MIN = 0.35
```

O que é: a solidez (também chamada de convexidade) mede quão “cheio” é um objeto em relação à sua **envoltória convexa** — o menor polígono convexo que envolve completamente o contorno (15):

$$\text{Solidez} = \frac{\text{Área do Contorno}}{\text{Área da Envoltória Convexa}}$$

Um círculo perfeito ou um quadrado têm $\text{solidez} = 1,0$. Uma estrela ou um “C” aberto têm solidez próxima de zero.

Por que é importante: parafusos vistos de cima ou de lado são objetos razoavelmente compactos — a cabeça hexagonal, cilíndrica ou Phillips tem solidez tipicamente entre 0,7 e 1,0. Artefatos de ruído e reflexos tendem a ter solidez muito baixa ($< 0,4$).

Atenção: parafusos vistos de lado (perfil) podem ter solidez mais baixa que os vistos de cima. Se as imagens contêm parafusos em diferentes orientações, mantenha `SOLIDEZ_MIN` em torno de 0,35.

9.3 Razão de Aspecto (`ASPECTO_MIN` e `ASPECTO_MAX`)

```

1 ASPECTO_MIN = 0.10
2 ASPECTO_MAX = 10.0

```

O que é: a razão de aspecto (*aspect ratio*) é a proporção matemática ortogonal entre a largura e a altura do retângulo mínimo alinhado aos eixos cartesianos (*bounding box*) que envolve o contorno (6):

$$\text{Aspecto} = \frac{\text{Largura}}{\text{Altura}}$$

Um objeto quadrado tem aspecto = 1,0. Um objeto muito largo tem aspecto $\gg 1$. Um objeto muito alto tem aspecto $\ll 1$.

Por que é importante: parafusos vistos de cima têm aspecto próximo de 1,0 ($\pm 20\%$). Parafusos vistos de lado podem ter aspecto de 3 a 8. O filtro elimina objetos extremamente alongados (linhas, fios) ou achatados (reflexos horizontais).

9.4 Extent (EXTENT_MIN)

```

1 EXTENT_MIN = 0.15

```

O que é: o descritor de preenchimento espacial *extent* mede a proporção exata entre a área real calculada do contorno e a área total do seu retângulo delimitador (7):

$$\text{Extent} = \frac{\text{Área do Contorno}}{\text{Largura} \times \text{Altura da Bounding Box}}$$

Um quadrado perfeito alinhado com os eixos tem *extent* = 1,0. Um círculo perfeito tem *extent* $\approx 0,785$. Formas em “L” ou triângulos têm *extent* baixo.

Por que é importante: o *extent* complementa a solidez. Enquanto a solidez compara o objeto com sua forma convexa ideal, o *extent* compara com o retângulo que o contém. Objetos muito espalhados ou com muitos “braços” têm *extent* baixo.

Relação entre Solidez e Extent:

- Alta solidez + baixo *extent*: objeto compacto mas mal alinhado com os eixos (ex.: parafuso rotacionado 45°).
- Baixa solidez + alto *extent*: objeto com recortes internos mas dentro de um *bounding box* estreito.
- Ambos baixos: fragmento irregular — quase certamente não é um parafuso.

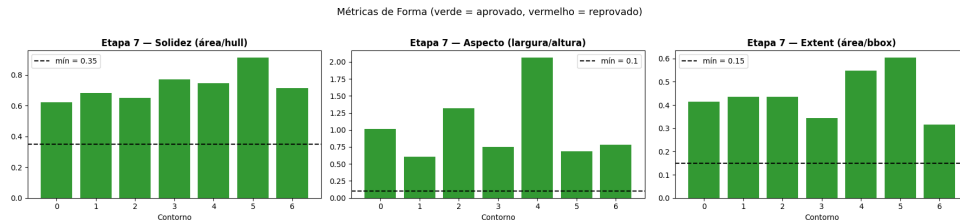


Figura 10: Distribuição dos Filtros de Forma.

10 Conclusão

Este relatório descreveu em detalhe cada etapa do pipeline de contagem de parafusos por visão computacional, desde a conversão para escala de cinza até os filtros de forma aplicados sobre os contornos detectados. Ao longo das seções, foram apresentados os fundamentos teóricos de cada operação, os parâmetros configuráveis e critérios objetivos para ajustá-los conforme o cenário de uso.

Desempenho observado

A abordagem demonstrou bom desempenho no cenário para o qual foi primariamente calibrada: parafusos centralizados na imagem, sem sobreposição entre si, sem sombras pronunciadas e sem objetos estranhos no campo de visão. Nessas condições, a combinação de binarização adaptativa, morfologia e filtros de forma é capaz de isolar e contar cada parafuso com precisão satisfatória.

Por outro lado, o pipeline apresenta limitações relevantes em cenários mais adversos:

- **Iluminação irregular:** variações abruptas de brilho entre regiões da imagem podem fazer com que a binarização adaptativa produza artefatos, mesmo com o ajuste de `blockSize` e `C`, comprometendo a qualidade da máscara binária.
- **Objetos não-parafuso:** ferramentas, sujeira, reflexos ou outros elementos com forma semelhante à dos parafusos podem passar pelos filtros de solidez, aspecto e *extent*, gerando falsos positivos difíceis de eliminar sem estreitar os limiares a ponto de rejeitar parafusos válidos.
- **Sobreposição entre parafusos:** mesmo com o algoritmo Watershed ativo, parafusos com alto grau de sobreposição frequentemente não são separados de forma confiável, resultando em subcontagem. Esse é o principal ponto de falha do pipeline nas condições testadas.

Ajuste de parâmetros e testes realizados

Uma das vantagens da arquitetura adotada é a parametrização explícita de cada etapa. Dependendo do cenário de captura — distância da câmera, tipo de parafuso, condições de iluminação — é possível reconfigurar os parâmetros descritos neste relatório para melhorar o desempenho sem alterar a estrutura do pipeline.

Durante o desenvolvimento, foram conduzidos testes com diferentes combinações de parâmetros sobre o conjunto de imagens disponível. A configuração final obteve bons resultados gerais: lidou adequadamente com sombras moderadas, suprimiu a maioria dos falsos positivos por meio dos filtros de forma e separou corretamente parafusos próximos em boa parte dos casos. A sobreposição total entre objetos, no entanto, permaneceu como um problema sem solução completa dentro da abordagem clássica, mesmo com o Watershed devidamente configurado.

Limitações da abordagem e perspectivas futuras

A principal restrição enfrentada ao longo do projeto foi a quantidade limitada de imagens rotuladas disponíveis para calibração e validação. Com um conjunto maior e mais diverso de imagens anotadas, seria viável considerar abordagens baseadas em aprendizado profundo end-to-end, como arquiteturas YOLO (16), que tendem a generalizar melhor para cenários complexos de iluminação, oclusão severa e variação estocástica de pose.

Modelos de segmentação de instâncias de propósito geral em estado da arte baseados em Transformers, como a família *Segment Anything Model* (SAM) desenvolvida pela Meta AI (1), seriam candidatos naturais para substituir o processamento clássico, operando com exímia precisão em segmentação *zero-shot*. Contudo, essa classe de modelos impõe restrições computacionais severas e latência elevada, exigindo aceleração por hardware (GPU dedicada) em tempo real, o que contraria o requisito mandatório de execução local em arquitetura móvel restrita estabelecido para este projeto.

Dessa forma, dado o conjunto de restrições existentes, volume limitado de dados, ausência de GPU e necessidade de operação em *mobile*, o pipeline de visão computacional clássico apresentado neste relatório representa a melhor abordagem encontrada dentro das condições do desafio, oferecendo um equilíbrio racional entre precisão, robustez e custo computacional.

Etapa 8 — Resultado Final: 7 parafuso(s)

Figura 11: Resultado final após aplicação dos filtros.

11 Guia de Diagnóstico — O que fazer quando a contagem erra

11.1 Cenário A: Parafusos válidos sendo rejeitados

Ative `MODO_DEBUG = True` e observe o motivo escrito em vermelho sobre os contornos rejeitados.

Tabela 3: Diagnóstico — parafusos válidos rejeitados.

Motivo exibido	Parâmetro	Direção
area=150 (pequeno)	AREA_MIN	Diminuir
area=55000 (grande)	AREA_MAX	Aumentar
solidez=0.28	SOLIDEZ_MIN	Diminuir
aspecto=0.08	ASPECTO_MIN	Diminuir
aspecto=11.2	ASPECTO_MAX	Aumentar
extent=0.12	EXTENT_MIN	Diminuir

11.2 Cenário B: Ruídos e falsos positivos sendo contados

Tabela 4: Diagnóstico — falsos positivos na contagem.

Problema	Parâmetro	Direção
Muitos fragmentos minúsculos	AREA_MIN	Aumentar
Sombras grandes detectadas	AREA_MAX + SOLIDEZ_MIN	Diminuir + Aumentar
Fundo texturizado gerando detecções	BLUR_KERNEL + MORPH_KERNEL	Aumentar ambos
Objetos nas bordas contados	MARGEM	Aumentar

11.3 Cenário C: Parafusos agrupados contados como um só

Tabela 5: Diagnóstico — grupos contados como objeto único.

Problema	Solução
Watershed desativado	Ativar <code>USAR_WATERSHED = True</code>
Watershed ativo mas não separando	Diminuir <code>WATERSHED_DIST_LIMIAR</code> (0,3 $\rightarrow$ 0,2)
<code>CLOSE_KERNEL</code> muito grande	Reduzir ((9,9) $\rightarrow$ (5,5))

11.4 Cenário D: Parafuso único dividido em dois

Tabela 6: Diagnóstico — parafuso único segmentado incorretamente.

Problema	Solução
Watershed segmentando demais	Aumentar <code>WATERSHED_DIST_LIMIAR</code> (0,3 $\rightarrow$ 0,5)
Buraco interno grande no objeto	Aumentar <code>CLOSE_KERNEL</code> para fechar antes do Watershed
Formato do parafuso muito irregular	Aumentar <code>BLUR_KERNEL</code> para suavizar mais antes da binarização

Relatório gerado para o pipeline `contar_parafuso_batch.ipynb` — Desafio de Modelagem Preditiva.

12 Aplicação Web — Microserviço FastAPI

O pipeline de visão computacional descrito nas seções anteriores foi encapsulado como um microserviço web utilizando o framework **FastAPI**, expondo uma API REST assíncrona que permite integração com qualquer cliente capaz de realizar requisições HTTP.

Arquitetura

A aplicação segue uma arquitetura de camada única com separação clara entre interface e processamento: o módulo `app.py` é responsável exclusivamente pelo roteamento HTTP e validação de entrada, enquanto toda a lógica de visão computacional permanece isolada em `processing.py`. Essa separação facilita a manutenção independente de cada componente e permite substituir o frontend ou o pipeline sem impacto na outra camada.

Endpoint de processamento

O serviço expõe um único endpoint funcional:

```
1 POST /api/process
2 Content-Type: multipart/form-data
3 Campo: file (imagem)
```

O cliente envia a imagem como `multipart/form-data`. O servidor valida o tipo, executa o pipeline e devolve a imagem composta em PNG com dois cabeçalhos HTTP customizados:

- `X-Screw-Count` — quantidade de parafusos detectados
- `X-Rejected-Count` — quantidade de contornos rejeitados pelos filtros

Processamento assíncrono

Como o pipeline do OpenCV é puramente síncrono e computacionalmente intensivo, a chamada ao módulo de processamento é delegada a uma `thread` separada via `asyncio.to_thread`, evitando o bloqueio do *event loop* do servidor durante a execução. Isso permite que o serviço mantenha responsividade e aceite novas requisições enquanto uma imagem está sendo processada.

Interface web

Além da API, o serviço disponibiliza uma interface de upload em `GET /` que permite o uso direto pelo navegador, sem necessidade de cliente externo.



Figura 12: Print da aplicação desenvolvida para o desafio.

Referências

- [1] KIRILLOV, A. et al. Segment anything. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. [S.l.: s.n.], 2023. p. 4015–4026.
- [2] KRIZHEVSKY, A.; SUTSKEVER, I.; HINTON, G. E. ImageNet classification with deep convolutional neural networks. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2012. v. 25.
- [3] GONZALEZ, R. C.; WOODS, R. E. *Digital Image Processing*. 4th. ed. [S.l.]: Pearson, 2018.
- [4] BRADSKI, G. The OpenCV library. *Dr. Dobbs's Journal of Software Tools*, v. 25, n. 11, p. 120–123, 2000.
- [5] International Telecommunication Union. *Studio Encoding Parameters of Digital Television for Standard 4:3 and Wide-Screen 16:9 Aspect Ratios*. Geneva: [s.n.], 2011. Recommendation ITU-R BT.601-7.
- [6] PEDRINI, H.; SCHWARTZ, W. R. *Análise de Imagens Digitais: Princípios, Algoritmos e Aplicações*. São Paulo: Thomson Learning, 2008.
- [7] NIXON, M.; AGUADO, A. *Feature Extraction and Image Processing for Computer Vision*. 4th. ed. [S.l.]: Academic Press, 2019.
- [8] OTSU, N. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, v. 9, n. 1, p. 62–66, 1979.
- [9] SERRA, J. *Image Analysis and Mathematical Morphology*. [S.l.]: Academic Press, 1983.
- [10] SOILLE, P. *Morphological Image Analysis: Principles and Applications*. [S.l.]: Springer, 2013.
- [11] VINCENT, L.; SOILLE, P. Watersheds in digital spaces: An efficient algorithm based on immersion simulations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, v. 13, n. 6, p. 583–598, 1991.
- [12] ROSENFELD, A.; PFALTZ, J. L. Sequential operations in digital picture processing. *Journal of the ACM*, v. 13, n. 4, p. 471–494, 1966.
- [13] MEYER, F. Topographic distance and Watershed lines. *Signal Processing*, v. 38, n. 1, p. 113–125, 1994.
- [14] SUZUKI, S.; ABE, K. Topological structural analysis of digitized binary images by border following. *Computer Vision, Graphics, and Image Processing*, v. 30, n. 1, p. 32–46, 1985.

-
- [15] PREPARATA, F. P.; SHAMOS, M. I. *Computational Geometry: An Introduction*. [S.l.]: Springer, 2012.
- [16] REDMON, J. et al. You only look once: Unified, real-time object detection. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2016. p. 779–788.